Supervised Learning Experiment Report

Peixuan Wang

***Abstract*— This report is a submission to the required assignment of the Supervised Learning module, which contains the necessary contents about the experiments specified in the information document in Canvas.**

I. PROBLEM INTRODUCTION

A. Predict high-risk patients based on risk-factor data

The problem I choose for cancer dataset is to detect the high-risk patients based on the demographic information and risk factors. By building ML algorithms, we can alarm the patients who are inclined to become a high-risk individual so they can be fully prepared to face the problem in advance.

B. Predict company bankruptcy based on financial data

This data set contains relatively complete data that illustrate the financial status of companies and their operational status, such as being bankrupt or not. So I think it would be cool if I could build a model to predict whether a company would end up with bankruptcy since there are so many financial indexes of listed companies accessible to the public, and the patterns of a company's bankruptcy are not so easy to induct by human force. This kind of work can help people make good investments and also provide a nice way to prevent frauds.

II. HYPOTHESES

Based on my understanding of the three algorithms in the experimental objectives, I present the following hypotheses:

Firstly, regarding the KNN algorithm, I believe its performance on the two datasets will be relatively balanced. However, on the bankruptcy dataset, due to the extremely high dimensionality of this dataset, KNN is likely to encounter the curse of dimensionality.

Besides that, concerning the SVM algorithm. Without considering the characteristics of the dataset itself, the cancer dataset is primarily composed of binary variables, which may be well-suited for classification using a linear kernel SVM.

Additionally, SVM should also be cautious of situations with excessively high dimensions. In contrast to the cancer dataset, the neural network model should be more adept at handling the bankruptcy dataset, which mainly consists of continuous variables.

III. EXPERIMENTS ON CANCER DATASET

A. Data Exploration

This data set contains global cancer patient data reported from 2015 to 2024, designed to simulate the key factors that influence cancer diagnosis, treatment, and survival. It includes a variety of features commonly studied in the medical field, such as age, gender, type of cancer, environmental factors, and lifestyle behaviors. This dataset contains 50000 data points and 15 columns.

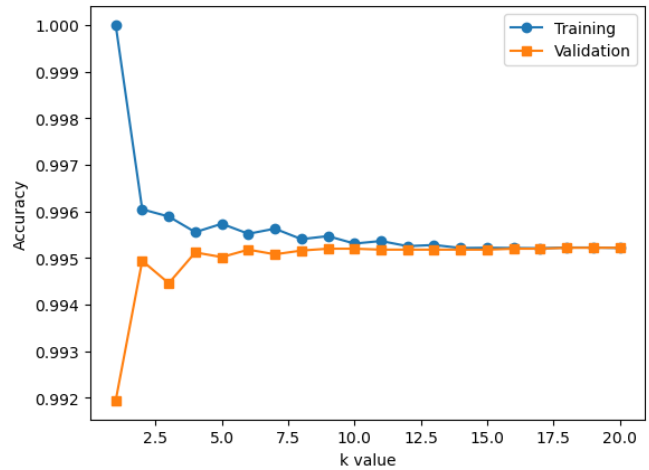

Fig. 1. KNN Performance on Various k Values

Since I want to predict potential high-risk patients by classifying them into two categories, high-risk or not, I need to define what's high-risk first.

What I have in mind is that the severity of patients' cancer is definitely a factor, but for a wide range of patients around the world, cancer is not the dilemma that defeats them first, but the bill is. For rich people, even if the situation seems bad, they can still take the state-of-the-art treatment no matter what, while for less fortunate families, even the mild kind of cancer might tear them apart. So, both of these factors need to be not optimistic to make a real high-risk patient.

The cost of treatment is the dataset is uniformly distributed, ranging from 5,000 to 100,000. And the median of the whole sample is about 50,000. The severity score is normally distributed with about 7 being its 95 percentile. So, I label patients whose cost of treatment is greater than 50,000 and severity score above 7.0 as high-risk patients.

B. Experiments on KNN Algorithm

1) Tuning: The most important parameter for KNN model is the k-value, so I tried a series of k-values to select a best model.

As we can see in Fig.1, the training accuracy and validation accuracy are getting close to each other with k value increasing. When k = 6, the two curves are already close enough, and after k exceeds 10, the accuracies on training set and validation set are approximately same. The model seems to have a nice accuracy and generalization. 10 seems to be a fair value.

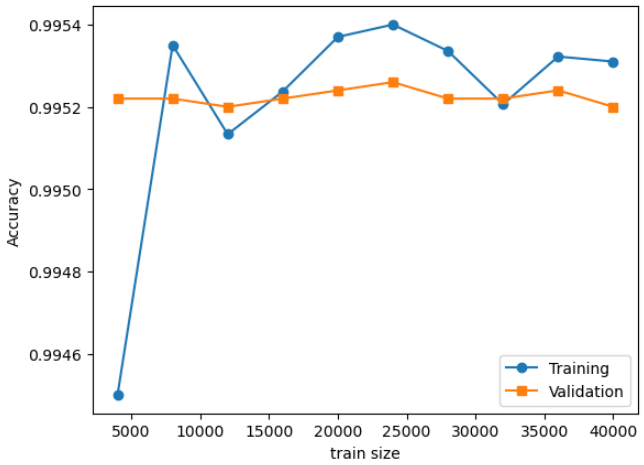

Fig. 2. Learning Curve of KNN when k = 10

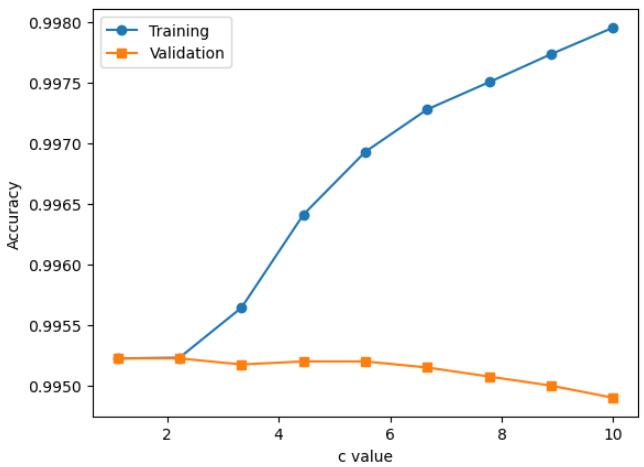

Fig. 3. SVM Performance on Various C Values

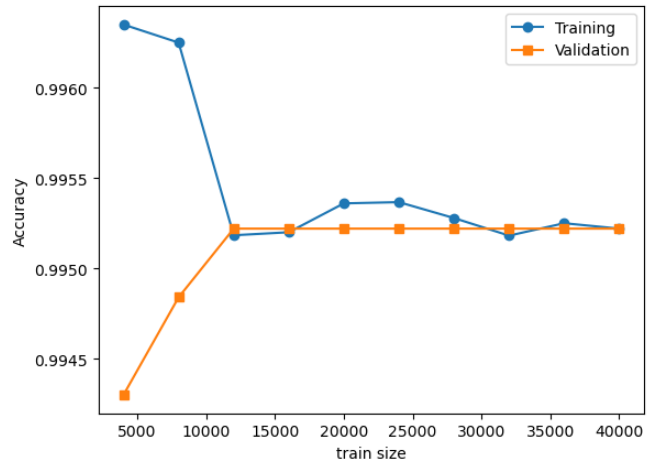

Fig. 4. Learning Curve of SVM using Linear Kernel

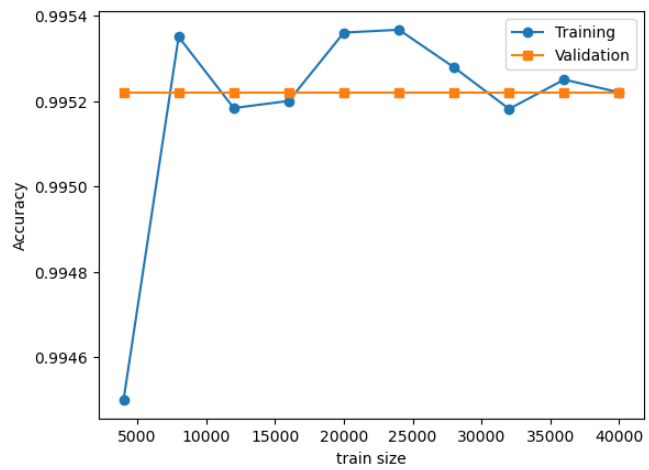

Fig. 5. Learning Curve of SVM using RBF Kernel

2) Learning Curve: In Fig.2, most of the time, the training score is higher than validation score, but sometimes it's otherwise. My guess is that since I introduce lots of dummy variables to represent features like gender, country and cancer state, my sample space is relatively narrow, so it somehow make extra data points like noises to this model, interfering its performance.

C. Experiments on SVM Algorithm

1) Tuning: The parameter of SVM that can to be tuned is C, which is a trade-off between making right classification and maximizing the gap between two classes.

Based on Fig.3, when C is below 2, the model works pretty well, two curves almost overlap, which suggests the model works similarly on both training and validation set, showing a nice generalization. But when C gets too large, these two curves dramatically separate. The model clearly overfits.

This is easy to explain because C represents how much I penalize making mistakes, instead of regularizing the model. So when I give SVM model a large penalty on making mistakes, it starts to only care about getting all instances in the training set right, not considering what will happen in the outside world, which appears as this bad performance on validation set. To make a balanced choice, I'll set C to be 1 in the following experiments.

2) Learning Curve: I tried two different kernels methods for comparison shown in Fig.4 and Fig.5.

The tendency of two kernels' corresponding curves are similar. And RBF kernel seems to improves faster. Probably its nonlinearity make it better at learning nonlinear rules.The final accuracies don't vary much thanks to the large amount of data, if the dataset is smaller, the performance of linear kernel might not be so good.

I accidentally notice that the learning curve of SVM using RBF kernel is somehow similar to that of KNN with large k value. I guess the reason is that they're both nonlinear learner based on local similarity and they're both sensitive to noises, so it's possible they tend to generalize similar decision boundaries.

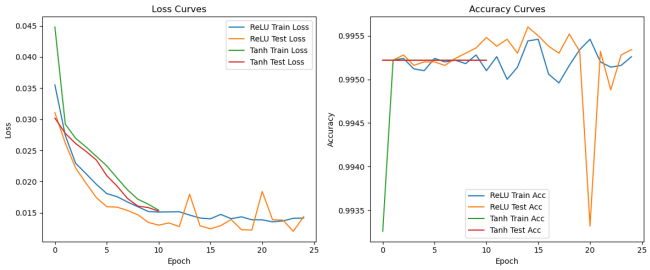

Fig. 6. Loss/Accuracy-Iteration Using Different Activations

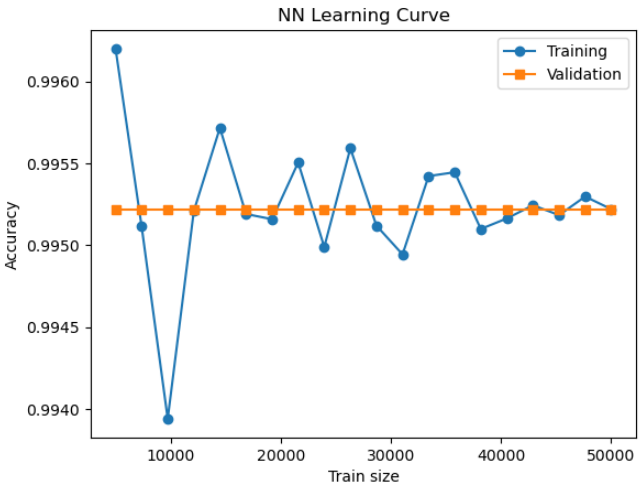

Fig. 7. Default ANN Performance

D. Experiments on Neural Nets

Considering that the number of input dimensions is 28, the default neural network structure used here has two layers, with 16 neurons in the first layer and 8 in the second layer.

1) Activation Comparison: I tried two different activation functions, the loss and accuracy curve of these two ANNs are shown in Fig.4.

The ANN using ReLU seems to be improving slightly faster, but also shows a strong sense of instability. It's accuracy and loss keep varying even after many iterations. When it comes to the 20th iteration, its performance on validation set even has a severe decline. It seems this model is very likely overfitting the data.

The model with Tanh activation shows a slightly slower descent in loss, but its curve is much smoother, and quickly stabilizes, triggering the early-stop mechanism way earlier than ReLU. However, it is also possible that gradient disappearance actually occurred.

2) Learning Curve: The learning curve of ANN using ReLU activation function is plotted in Fig.7. This curve has strong oscillation when data amount is small, but when it gets large, it gradually converge to the curve on validation set. I guess the reason is the training batch is set too small in the early stage, the validation scores look fine is only because they're averaged.

3) Extra Experiment: Comparison of Architectures: According to Fig.8, it is quite evident that wide models have a faster learning speed, while deep neural networks do not demonstrate a stronger fitting capability, possibly due to a relatively insufficient number of neurons.

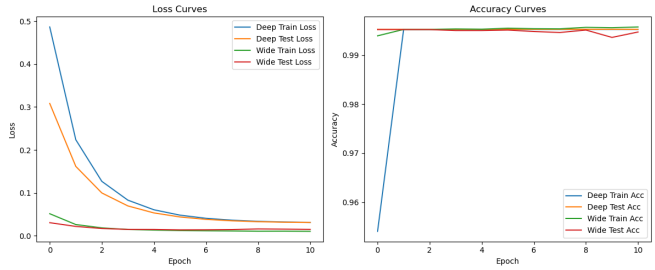

Fig. 8. ANN Performance of Different Architectures

4) Extra Experiment: Comparison of Initializations: I tried three initialization methods, and record their best test accuracy:

xavier initialization - Best Test Accuracy: 0.9953 he initialization - Best Test Accuracy: 0.9954 uniform initialization - Best Test Accuracy: 0.9952

From the training process data, the he method ultimately has stronger classification ability, while the uniform converges faster.

5) Extra Experiment: Comparison of Regularization: I tried four regularization methods, and record their best test accuracy:

None initialization - Best Test Accuracy: 0.9954 Dropout initialization - Best Test Accuracy: 0.9952 L2 initialization - Best Test Accuracy: 0.9952 Dropout+L2 initialization - Best Test Accuracy: 0.9952

The convergence speed of None initialization is significantly slower than others, which indicates that the regularization method does play a role in suppressing overfitting.The network with L2 initialization shows that the loss on the training set and test set almost overlaps, reflecting the best generalization ability.

IV. EXPERIMENTS ON BANKRUPTCY DATASET

A. Data Exploration

This dataset were collected from the Taiwan Economic Journal for the years 1999 to 2009. It contains 95 financial indicators commonly seen in financial statements like balance sheet, income statement and cash flow chart. It contains 6819 instances. Company bankruptcy was defined based on the business regulations of the Taiwan Stock Exchange.

Since it's acquired from Taiwan Stock Exchange, the label is already determined, whether this company is bankrupt. I don't need to construct a label by myself like I did in cancer dataset.

Since these financial indicators have all kinds of units, some even don't really have a unit. There are accumulated capital, cash flows, rates and ratios. So to make our algorithms easier to converge, and eliminate the effect of scale. I did standard scale to scale the data, so that we won't need to worry about the problems mentioned above.

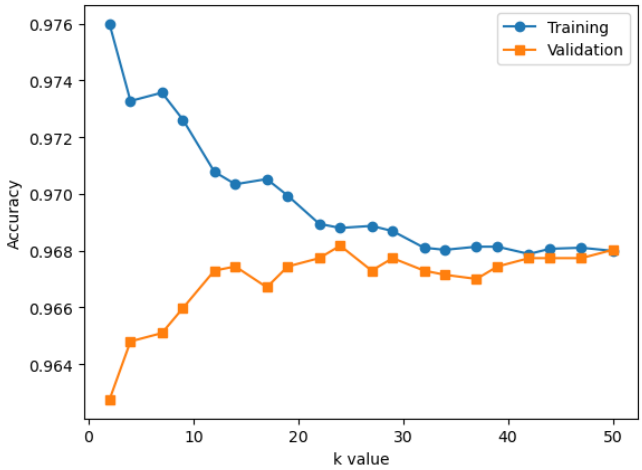

Fig. 9. KNN Performance on Various k Values

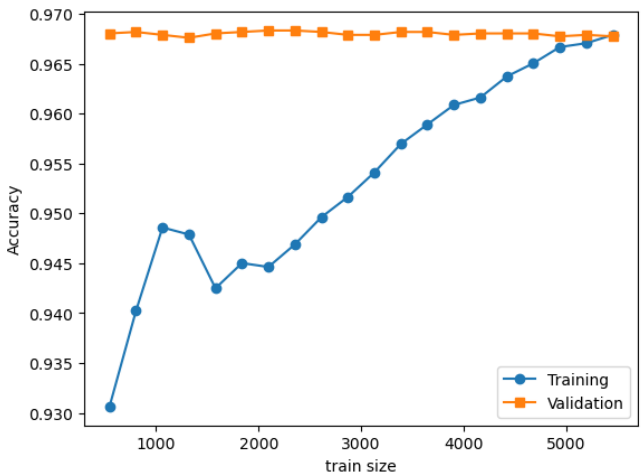

Fig. 10. Learning Curve of KNN when k = 42

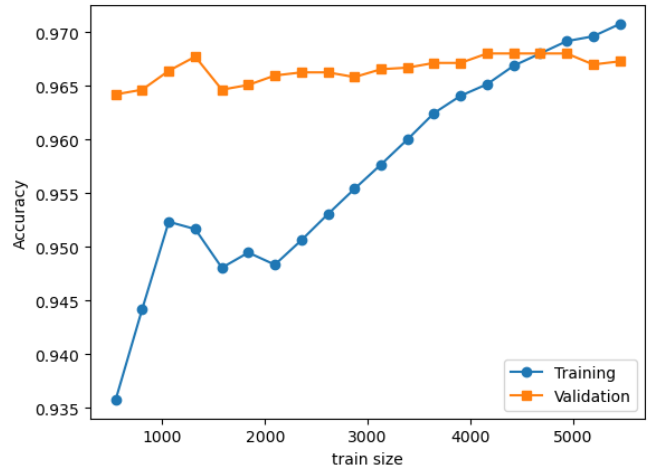

Fig. 11. Learning Curve of KNN when k = 12

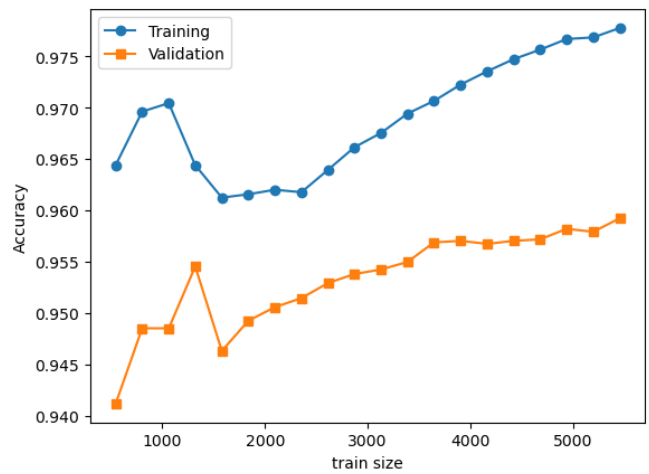

Fig. 12. Learning Curve of KNN when k = 12

B. Experiments on KNN Algorithms

1) Tuning: Like what I did earlier, I tried a series of k values, then I derived two curves shown in Fig.8.

The pattern is almost the same as that in Fig.1, except that the convergence of these two lines this time is slower. It seems like 42 is a fine k value.

2) Learning Curve: I have observed an interesting phenomenon. As we can tell from Fig.9. when data size is small, performance on the training set is even worse than that on the validation set. And when the data size grows larger, this abnormal phenomenon gradually disappeared. It seems quite confusing but I'd like to bring up an explanation.

42 is not only the ultimate answer, but also a remarkably large number for a k value, which means I need 42 points to vote to decide which class this point that I'm considering should fall into. But, when the data size is small, when doing cross validation, the folds used to train the model are also small. The 42 points that we use might not concentrate enough to make an accurate estimation even on training set. So we can suppose that with smaller k, the phenomenon will disappear.

So I tried another 2 k-values.

When k = 12, the model starts to have a better performance on training set, and when k = 3, the training curve seems really back to normal. But it doesn't necessarily suggest that I chose a bad k value earlier, because according to the validation curve, when I'm trying make the training curve look better, the validation curve is getting not as good as before.

C. Experiments on SVM Algorithm

1) Tuning: The pattern shown in Fig.12 is similar to that of Fig.3. But the only difference is this time two curves don't overlap even when c is 0, which suggests regularization term can not prevent some extent of overfitting. I'll still use C = 1 in the following experiments since that's a default value.

2) Learning Curve: The tendency of curves are also similar to what I get on the other dataset. But performance of RBF kernel is significantly better than linear kernel. This could result from the intrinsic characteristics of the dataset.

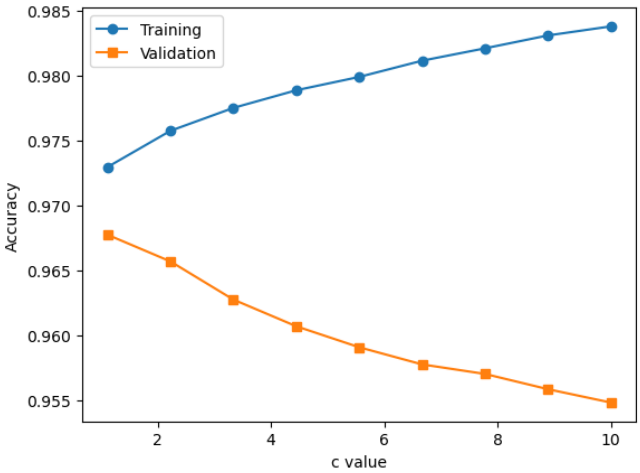

Fig. 13. SVM Performance on Various C Values

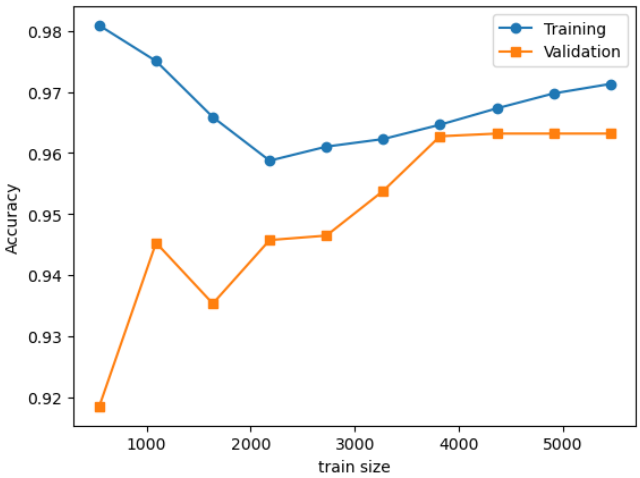

Fig. 14. Learning Curve of SVM using Linear Kernel

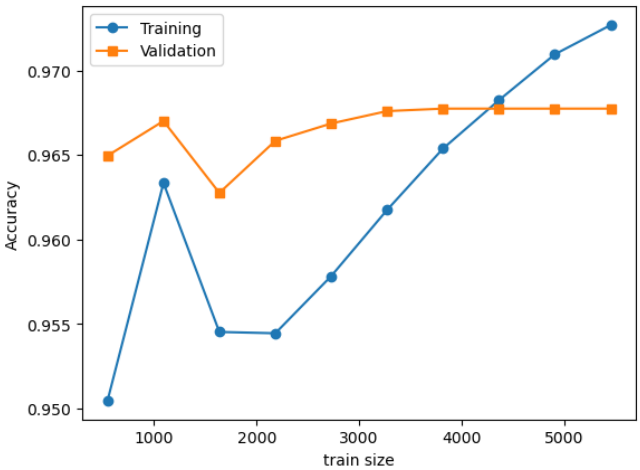

Fig. 15. Learning Curve of SVM using RBF Kernel

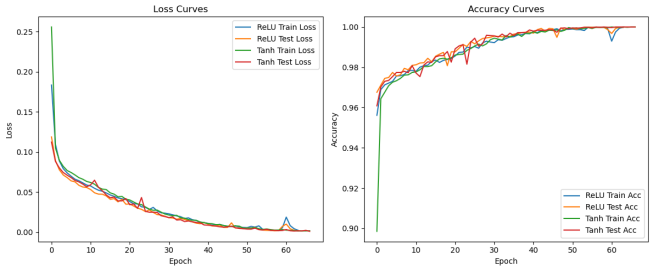

Fig. 16. Loss/Accuracy-Iteration Using Different Activations

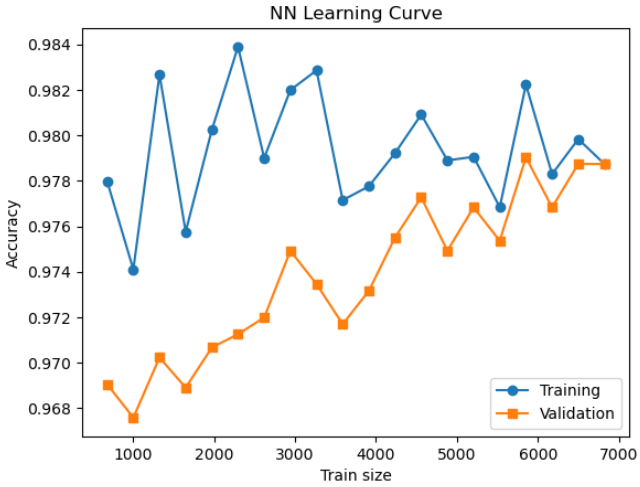

Fig. 17. Default ANN Performance

First, the data itself may be linearly separable or approximately linearly separable.Then, due to the fact that this dataset has up to 95 features while there are only 6000 rows, it is a very high-dimensional dataset. In this case, the RBF kernel has a high risk of overfitting, which may also be one of the reasons why its performance is inferior to that of the linear kernel.

D. Experiments on Neural Nets

This time, considering that the number of input dimensions is 95, the scale of the network has been increased. The default neural network structure used has two layers, with the first layer having 64 neurons and the second layer having 32.

1) Activation Comparison: Unlike ANN models we trained on cancer dataset, these models take much more epochs to stop training, that's because this time we have way more neurons to train. And the improving processes are also more stable in, as shown in Fig.15, which suggests more neurons might bring better performance.

The difference between curves using ReLU and Tanh is not obvious anymore. Though, I can still notice that Tanh changes a little bit slower than ReLU, which is still consistent with the finding on the other dataset.

As the dataset increases, the two curves gradually converge, noting that the fluctuation trends of the two lines are roughly the same. Therefore, I believe that these fluctuations

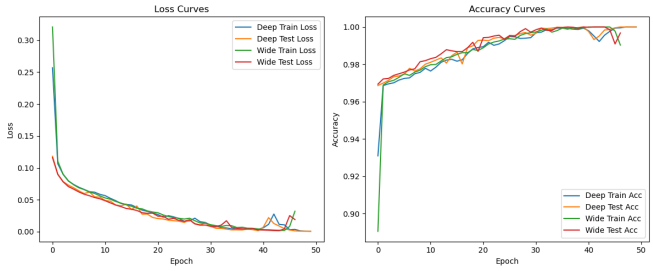

Fig. 18. ANN Performance of Different Architectures

may be caused by the randomness of the sample batches chosen during the model training process, which is systematic, and thus the performance tendency on both datasets remains consistent.

2) Extra Experiment: Comparison of Architectures: In these model, whose learning curves are plotted in Fig.18, an increase in width or the number of layers did not result in noticeable changes, possibly because the effect of increased number of neurons overshadowed the impact of the architecture.

3) Extra Experiment: Comparison of Initializations: I tried three initialization methods, and record their best test accuracy:

xavier initialization - Best Test Accuracy: 1.0000 he initialization - Best Test Accuracy: 1.0000 uniform initialization - Best Test Accuracy: 0.9999

The results are basically consistent with those on the cancer dataset.

4) Extra Experiment: Comparison of Regularization: I tried three initialization methods, and record their best test accuracy:

None initialization - Best Test Accuracy: 1.0000 Dropout initialization - Best Test Accuracy: 0.9959 L2 initialization - Best Test Accuracy: 0.9987 Dropout+L2 initialization - Best Test Accuracy: 0.9922

The results are basically consistent with those on the cancer dataset, but the L2 initialization doesn't perform as well on this one.

V. CONCLUSIONS

This research report systematically explores the performance differences of three supervised learning algorithms (KNN, SVM, and neural networks) on two typical data tasks: risk assessment of cancer patients and bankruptcy prediction for companies. The experimental results reveal an important correlation between algorithm performance and dataset characteristics, providing a basis for algorithm selection in practical application scenarios.

Overall, neural networks have shown the highest predictive accuracy on both types of datasets, particularly with a more pronounced advantage in corporate bankruptcy prediction tasks. This exceptional performance stems from the ability of neural networks to abstract features hierarchically, allowing them to effectively capture complex patterns in high-dimensional data. Meanwhile, experiments revealed that different algorithms respond differently to specific datasets: in the cancer dataset, SVM demonstrated a unique advantage in stability, with its predictive accuracy remaining largely unaffected by changes in the size of the training data; whereas, the performance of neural networks on this dataset fluctuated significantly as the amount of data increased. This comparison highlights the differences in sensitivity to data distribution characteristics among algorithms.

The experimental results of the bankruptcy prediction data bring some important findings. This high-dimensional dataset exposes the limitations of traditional algorithms: KNN requires an extremely large k value (k=42) to achieve optimal accuracy, which significantly reduces computational efficiency; meanwhile, SVM struggles to find a suitable regularization parameter C, indicating that this algorithm faces adaptation challenges when dealing with complex financial data structures. These phenomena validate the practical constraints of 'curse of dimensionality' on traditional algorithms in high-dimensional data scenarios.

In terms of algorithm optimization, the research has revealed several technical insights: the choice of activation function in neural networks directly affects the training dynamics; while ReLU can accelerate convergence, it increases volatility, whereas Tanh provides a smoother learning process; regularization strategies vary in effectiveness across different datasets; Dropout and L2 effectively suppress overfitting in cancer data, but have limited effectiveness in bankruptcy data where accuracy is already near perfect; the choice of kernel function also requires weighing pros and cons; although the RBF kernel generally outperforms the linear kernel, it presents a higher risk of overfitting in high-dimensional financial data.

These findings are inspiring for practical applications. In the field of medical risk prediction, SVM should be prioritized to ensure model stability, especially when data collection is in a dynamically expanding phase; in financial risk warning scenarios, the computational cost of neural networks should be accepted in exchange for their outstanding predictive accuracy, which is critical for investment and financing decisions. Future research may explore hybrid modeling schemes that integrate the advantages of multiple algorithms and continue to deepen the understanding of algorithm adaptability in high-dimensional data scenarios, which will help develop more robust and effective predictive systems.